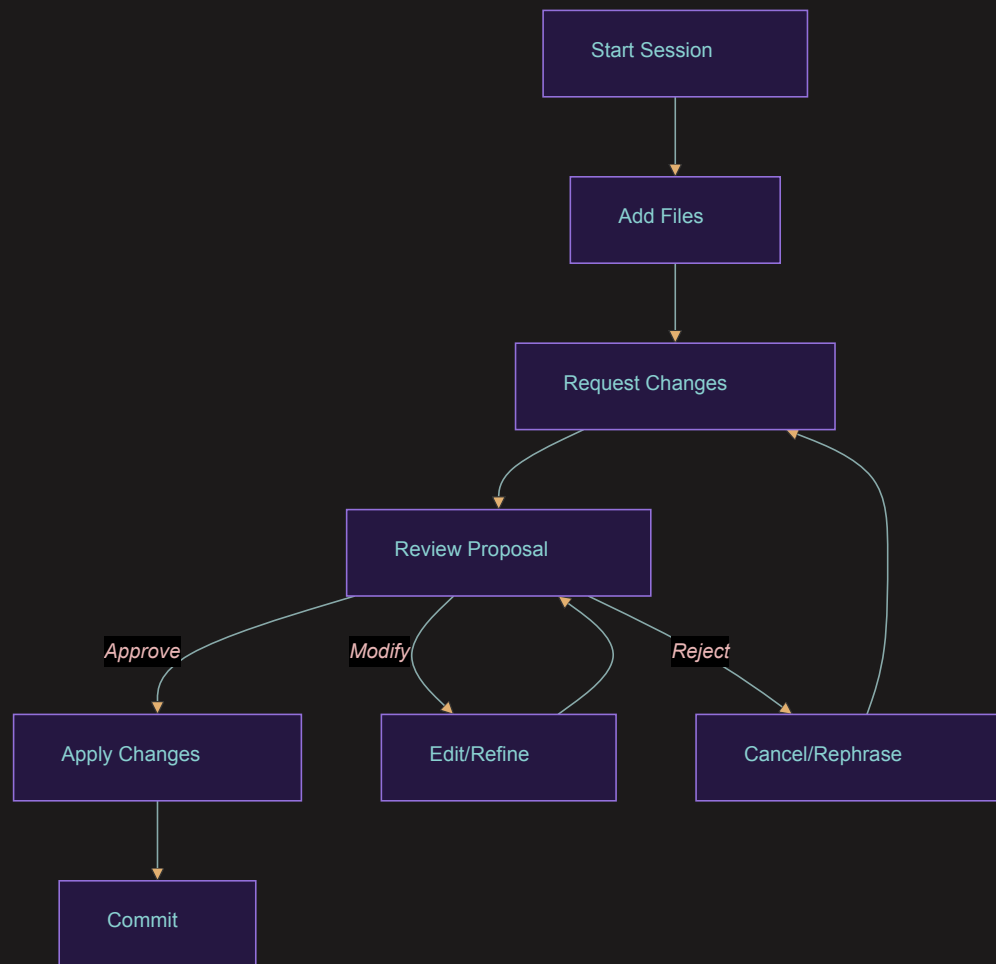


AI-Assisted Software Development

Module 3: Working with aider

Mike Burton

The aider Workflow



Effective Communication

DO:

- Be specific
- Provide context
- Break down complex tasks
- Reference file names
- Mention function names

DON'T:

- Be too vague
- Assume knowledge
- Request multiple unrelated changes
- Skip important details

Example:

```
> In user_auth.py, add password strength validation  
to the validate_password function. It should:  
- Require minimum 8 characters  
- Include at least one number  
- Include at least one special character
```

Working with Code Blocks

Different ways to reference code:

1. Direct Reference:

> Update the MAX_USERS constant in config.py to 100

2. Code Block Reference:

> Replace the sort_users function with this implementation:

```
def sort_users(users, key='name'):
    return sorted(users, key=lambda x: x[key])
```

3. Partial Updates:

> Add error handling to the process_data function around the file reading operation

Pattern Recognition

aider understands common coding patterns:

1. Function Modifications:

- > Add input validation to `process_user_data`

2. Error Handling:

- > Wrap the database operations in try/except blocks

3. Documentation:

- > Add docstrings to all functions in `auth.py`

4. Refactoring:

- > Extract the email validation logic into a separate function

Working with Multiple Files

Example: Updating related files

- > Add a new user role system:
 - 1. Add Role enum in models.py
 - 2. Update User class to include role
 - 3. Modify authentication in auth.py

aider will:

1. Analyze dependencies
2. Make coordinated changes
3. Maintain consistency
4. Commit all changes together

Code Review Patterns

When reviewing aider's changes:

1. Check Logic:

- Is the implementation correct?
- Are edge cases handled?
- Is error handling appropriate?

2. Check Style:

- Does it match project style?
- Is it well-documented?
- Is it readable?

3. Check Integration:

- Do changes work with existing code?
- Are imports handled correctly?
- Are dependencies updated?

Iterative Development

Example workflow:

1. Initial Request:

- > Create a function to validate email addresses

2. Refine Implementation:

- > Add support for custom domains

3. Add Features:

- > Add blacklist checking for spam domains

4. Optimize:

- > Cache the domain validation results

Working with Tests

aider can help with testing:

1. Creating Tests:

> Write unit tests for the email_validator function

2. Updating Tests:

> Update tests to cover the new domain blacklist feature

3. Test Fixes:

> Fix the failing test in test_email_validator.py

Error Handling

Common scenarios and solutions:

1. Unclear Response:

> Can you clarify what you mean by custom domains?

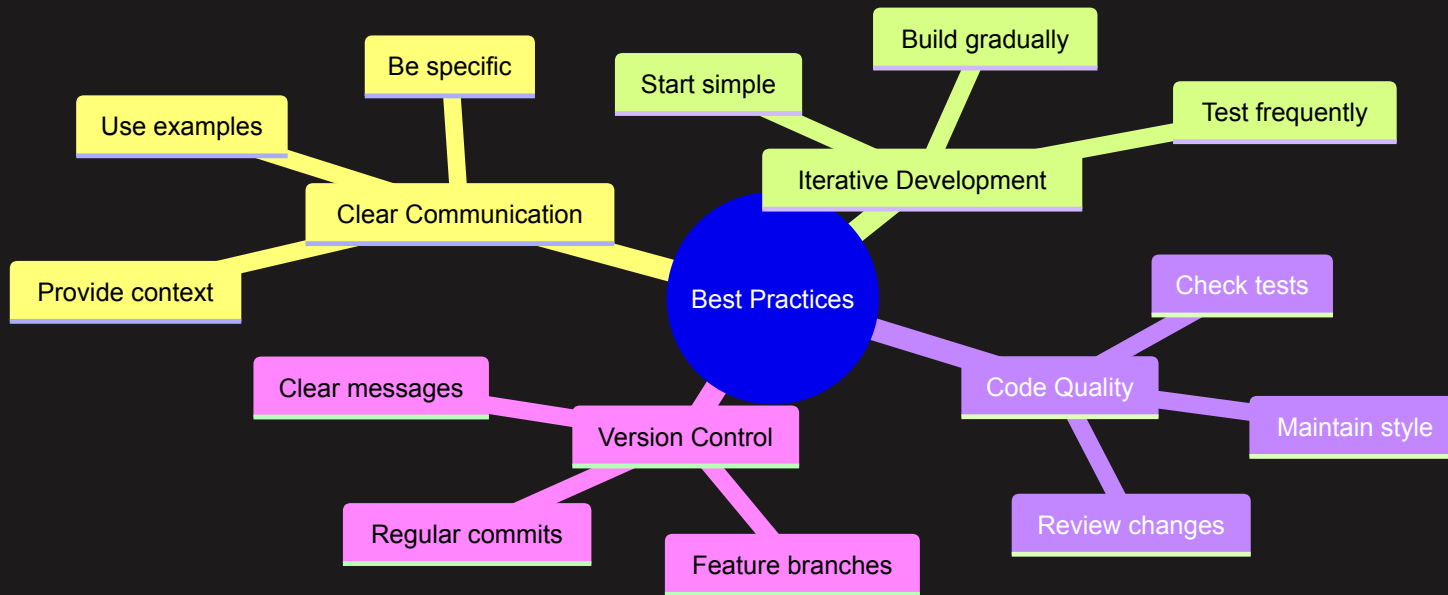
2. Incorrect Changes:

> That's not quite right. The validation should check for the @ symbol first

3. Partial Success:

> The domain validation looks good, but we also need to check for valid usernames

Best Practices



Common Patterns

1. Code Generation:

- > Create a new class for handling API requests

2. Refactoring:

- > Extract this logic into a separate utility function

3. Documentation:

- > Add type hints and docstrings to this class

4. Bug Fixing:

- > Fix the edge case where `user_id` is `None`

Advanced Communication

1. Providing Context:

> We're using SQLAlchemy for our ORM. Update the User model to include these new fields...

2. Referencing Standards:

> Following PEP 484, add type hints to all functions

3. Explaining Requirements:

> Due to GDPR requirements, we need to add data encryption to these user fields...

Recap: Working with aider

- Understanding core concepts
- Following effective workflows
- Communication patterns
- Working with code
- Handling multiple files
- Best practices
- Troubleshooting

Next Steps

- Exploring advanced features
- Custom configurations
- Integration with tools
- Complex refactoring

Questions to Consider:

- How can you improve your communication?
- What patterns work best for your projects?
- How can you optimize your workflow?